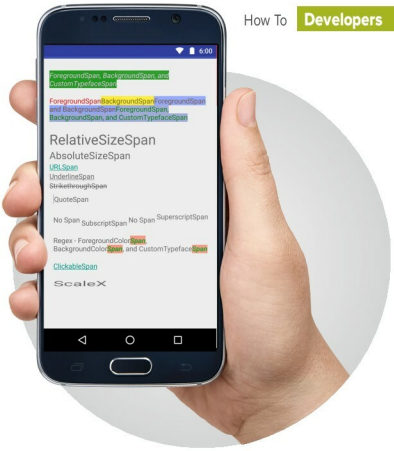
Exploring Spans for Fancy Text on Android



Android developers and DIY enthusiasts can use fancy text in their apps and games. In Android, text is not a matter of just typing it in and applying styles to it. It is an involved procedure that uses TextView, which contains Spans as classes. Read on to discover the magic of creating attractive text in Android.

Text is probably the most common thing in any app. There could hardly be an app without any text in it, and the easiest way to display text in Android is to use TextView from the Android framework. Most people might think that TextView is a pretty easy and lightweight widget, because it only draws text. And that it is probably just *canvas.drawText()* with some customisation for text like underline and bold. But it is a lot more complex than that. When we looked at the code of TextView, we found that it consists of more than 10,000 lines of code.

The packages that TextView primarily depends on to get things done contain the package *android.text.style*. Now, if you look at the classes or interfaces contained in this package, these are mostly Spans, which are used to attach mark-up or styling information to a piece of text. There are two main direct implementations of Span—Spanned and Spannable. Spanned is immutable and Spannable is mutable; otherwise, they mostly provide the same functionalities. The Spanned interface implements CharSequence, so you can set it to TextView with the setText method. During our regular development flow, we don't really deal with CharSequences but mostly work with Strings and StringBuilders. To help with that, the framework provides SpannableString and SpannableStringBuilder, both of which behave similar to String and StringBuilder, respectively, and also know how to handle Spans inside a String.

Now before going into building our own Spans and applying them for our custom styles, let's look at the Spans that the framework provides. These are AbsoluteSizeSpan, BackgroundcolourSpan, ClickableSpan and LocaleSpan, just to name a few. The framework provides us with a huge variety of Spans to style text. Let's talk about some of the most common ones which can help deal with text in day-to-day development workflows.

Figure 1 shows how the demo will look after we have done all the work. We will explain all the Spans according to the order shown in Figure 1. You can find the link to the source code of the demo application in the *References* section of the article.

RelativeSizeSpan

First, we're going to talk about one of the easiest Spans, which is RelativeSizeSpan. It is used to resize the text size relative to its current size. We will make a SpannableString with the text and then apply Spans to the specific range of characters in the string. This will be our regular pattern for applying Spans to our text in this article.